

Simulating LLaMA-3 on an Event-Driven TPU Simulator

Amaan Mohammed
University of California, San Diego
La Jolla, CA, USA
a5mohammed@ucsd.edu

Nikhita Neelakanta
University of California, San Diego
La Jolla, CA, USA
nneelakanta@ucsd.edu

Tejus Singh
University of California, San Diego
La Jolla, CA, USA
t1singh@ucsd.edu

Abstract—P3 is a C++ cycle-level, event-driven simulator for heterogeneous LLM inference accelerators. The simulated hardware consists of a weight-stationary systolic array, vector cores, access cores, and DMA engines connected to an HBM hierarchy. The simulator is timing-only: each unit computes a closed-form cycle latency for every operation, schedules a completion event, and signals downstream dependents. A programmatic LLaMA-3 schedule generator drives the simulator, emitting a full transformer forward pass with FlashAttention-2 causal block-skipping and Grouped-Query Attention (GQA). The 8B model matches the published parameter count to within rounding, and every MAC total agrees with independent hand derivations to within 0.01%. Sweeps across systolic array dimensions (32×32 to 512×512), HBM bandwidths (0.10–3.35 TB/s), prompt lengths (128–8192 tokens), and KV-head counts (1–32) consistently show that LLM decode is memory-bound on every studied configuration. Decode throughput tracks HBM bandwidth closely: the $26.9 \times$ gap between datacenter and edge decode rates mirrors the $33.5 \times$ bandwidth ratio, far more than the $64 \times$ array-area ratio.

I. INTRODUCTION

Before any application-specific integrated circuits (ASICs) are committed to silicon, designer must have an idea of how an target workload will run on the proposed chip’s architecture. Questions such as where time is most spent, potential compute or memory bottlenecks, and which adjustable parts of the architecture (unit count, bus widths, memory latency, etc.) change these questions. Building register-transfer logic to answer these questions is expensive and time-consuming. A simulator can take architecture descriptions and example workloads and return important metrics such as computational latency, memory bandwidth, utilization rates, etc. to answer architecture question early and guide the design process for an ASIC.

This report describes P3, our event-driven simulator and the LLaMA-3 workload mapped to and ran on it. An event-driven simulator represents work as events where each carries an intrinsic latency and dependencies on other events. The simulator advances time directly to the next event, so idle gaps between events costs no time. For an architecture and timing study for models like LLaMA3 where prefill and decode can expand to millions of operations, this event-based simulator style is practical for a design space sweep. Section 2 covers the simulator and engine, Section 3 covers the simulated workload (LLaMA-3, FlashAttention-2, Group-

Query Attention, and schedule generation), and lastly Section 4 covers hardware configurations, sweep results, and model results.

II. EVENT-DRIVEN SIMULATOR AND ENGINE

A. Simulator Overview

P3 is a discrete-event simulator. The core of the simulator is a min-heap of events which are keyed based on (cycle, seq_id). Cycle is the dominant key such that the earliest simulated time always pops off the min-heap first, with "seq_id", a timestamp given to events when they enter the simulator engine, breaking cycle latency ties between events. There are also "type" (OP_START, OP_DONE, etc.) parameters and "target unit" parameters used to help with event ordering. The event run loop pops the earliest event, advances the tracked global clock to its cycle, and then dispatches it on the target unit (src/core/event engine.cpp, with the Event struct in src/core/event.h). The main loop works as follows: while the event queue has elements, pop the top, set the "now" parameter to its cycle, and hand the event over to its target unit’s **handle** method. Contention for units is not resolved within the min-heap. Before an event is pushed and started with **OP_START**, the operation’s handler calls **reserve_unit_pool**, which finds the earliest free unit and resolves what the operation’s start cycle can be.

This simulator is timing only . It computes how many cycles an operation should take, and schedules **OP_DONE** that many cycles later, while notifying other events. This abstraction allows for fast simulation and study of architecture choices, bottleneck, and utilization.

B. Execution as Dependency Graph

A given schedule is a list of instructions, where each instruction contains fields such as "id", "op", "unit", "params", and "depends_on". This list is constructed into a directed, acyclic graph (DAG). The scheduler (src/schedule/scheduler.cpp) computes each node’s in-degree and a list of its successors. These computations are then stored as 3 contiguous vectors (dependencies, successors, IDs to instruction type), which allows the storing of large schedules like LLaMA3 which can be millions of instructions long. After this DAG is created, at run time the scheduler issues every zero-dependency node, where completion of an

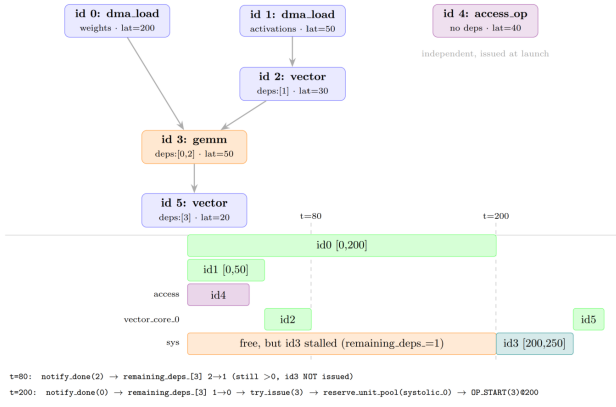


Fig. 1. Dependencies and scheduling on a six-instruction graph. Top: the schedule as a DAG, each node labeled with its operation, dependency list, and latency. Bottom: the resulting timeline across the DMA, access, vector, and systolic units. The id3 cannot issue until both of its producers complete; its counter reaching zero at $t=200$ triggers try issue, unit pool reservation, and then OP START.

event decrements that event’s successors’ counter. The successor event is issued when this counter turns to zero. Kahn’s algorithm is ran to validate DAG through **Schedule::validate** (`src/schedule/schedule.cpp`). Running overlapping events such as staging tile $i+1$ while tile i is being computed is an emergent feature of this zero-dependency instruction issue and unit pools. An example flow of execution is shown in Figure 1.

C. Unit Pools and Reservation

Units of the same type form a pool, with each unit in a pool getting an index suffix (`vector_core -i`, `vector_core_0`). When an operation issues, **reserve_unit_pool** (`src/core/event_engine.cpp`) scans the target unit’s pool and books the unit that becomes free the earliest. The particular unit then becomes “booked” for the duration of “busy cycles” dictated by the event latency. This reservation style also accounts for buffer and SRAM capacities. Units which are operating on full SRAM are skipped or stalled (ex. DMA units). This reservation model is what lets the schedule be transformed into overlapped layers of “concurrent” execution.

D. Base Hardware Units

For P3, five unit types are modeled, each units latencies’ are variable through the use of the yaml file under **configs/**:

Systolic Array (`src/units/systolic_unit.h`): This is the heart of the TPU simulator. It models the holding of a $K \times N$ weight block in the array while M rows of activation stream through. Fragmentation is over K and N , while M simply stream and never fragments. With weight double-buffering the per N -tile compute latency cost is:

$$\text{lat} = \left\lceil \frac{N}{\text{cols}} \right\rceil \left(w_{\text{load}} + \left\lceil \frac{K}{\text{rows}} \right\rceil \max(w_{\text{load}}, M) + \text{fill} \right)$$

Where $\text{fill} = (\text{rows}-1) + (\text{cols}-1)$, as the time for the element wavefront to cross the array. w_{load} defaults to **rows**.

This models behavior that a larger systolic array requires $\text{ceil}(K/\text{rows})$ weight loads. An available bidirectional fill option halves the fill term (hardware fills array from both sides) and optional code for an output-stationary data flow is available (though not validated).

DMA Engine (`src/units/dma_unit.h`): This unit involves two paths. The first is the HBM_transfer latency, where data is readied from off-chip after $\text{hbm_lat} + \lceil \text{bytes}/(\text{bw} \cdot \text{channels}) \rceil$ cycles.

Next, is the on-chip staging of an operand tile from SRAM to the array. This is modeled as: $\lceil \text{elements}/\text{rows} \rceil$ cycles. One column of row elements is processed per cycle over the operand bus.

Vector Core (`src/units/vector_unit.h`): This represents a SIMD lane that does reduction and other layer work such as RMSNorm, softmax, SiLU. The formula used is as follows:

$$\text{passes} \cdot \lceil \text{elem}/\text{simd} \rceil + \text{exp_ops} \cdot \text{exp_lat} \cdot \lceil \text{elem}/\text{simd} \rceil$$

The formula splits an op’s cost into cheap and expensive work over the same $\text{ceil}(\text{elem}/\text{simd})$ groups (the number of SIMD-width chunks needed to cover all elements): the first term, $\text{passes} \cdot \text{groups}$, charges 1 cycle per group for each simple linear pass (e.g., a row-max scan, a normalize pass, ordinary ALU ops that complete in one cycle per SIMD lane-group). The second term, $\text{exp_ops} \cdot \text{exp_lat} \cdot \text{groups}$, accounts separately for more complex operations (like exp in softmax), which take **exp_lat** cycles per group instead of 1, so each exp pass costs **exp_lat** times more than a regular pass, added on top of the base cost.

Access Core (`src/units/access_unit.h`): This units models the handling of transposes, copying, buffer fills at a latency of $\lceil \text{elements}/\text{bandwidth} \rceil$ per operation

Buffer: A double-buffered IBUF and OBUF SRAM model is defined for modeling scratchpad memory capacity. The on-chip working data is tracked through a shared capacity tracker (`src/core/event_engine.cpp`) that for example, a unit like the systolic unit can acquire at its **OP_START** and release at its **OP_DONE**.

E. Additional Modeled Features

Several features are included to make the simulation more realistic to modern TPUs; each lives in a latency formula or in the schedule structure, and each is a YAML toggle.

- **Weight double-buffering** (`src/units/systolic_unit.cpp`). Models the TPU weight FIFO: the next weight block is prefetched behind the current stream, so only the first load of a GEMM is ever exposed.
- **HBM pipelining** (`src/schedule/op_handlers.cpp`, `src/config/arch_config.h`). With `hbm_pipelined` on, each DMA only occupies its channel for bytes/bw cycles (not the full latency+bytes/bw), while each individual transfer still completes (**OP_DONE**) after the full

latency+bytes/bw, so back-to-back transfers’ latencies overlap.

- **SRAM operand and KV-prefetch double-buffering** (`src/schedule/tiler.cpp`, `src/schedule/llama_schedule.cpp`). Two ping-pong staging banks are expressed, so writing tile $i+1$ overlaps the GEMM on tile i (reading) and the next KV tile prefetches behind the current one, all using different banks.
- **SRAM capacity** (`src/core/event_engine.cpp`). With `model_sram` on, the IBUF and OBUF capacities are enforced. Working sets that exceed them spill to HBM and pay off-chip penalties in their latency.
- **Structural K-tiling** (`src/schedule/tiler.cpp`). An optional mode where the Tiler splits a $K > \text{rows}$ GEMM into explicit per-K-block sub-GEMMs and accumulate operations, modeling partial-sum traffic to OBUF rather than folding K into a large single latency term.

III. THE SIMULATED WORKLOAD AND SCHEDULE SOFTWARE

The simulator is driven by a schedule which operation should run, which abstract hardware pool should execute it, which numeric parameters define its latency, and which earlier instructions must complete before it can issue. In the C++ representation this is an `Instruction` with an `id`, `op`, `unit`, `params`, and `depends_on` list. The simulator never needs token values, activations, or weights. It needs only quantities such as GEMM dimensions, vector lengths, byte counts, and dependency edges. That separation makes the workload layer reusable: a new architecture or model variant can be represented by changing the schedule builder and YAML configuration while leaving the event engine, unit reservation, and roofline accounting intact. The concrete instruction representation and the YML parser are in `src/schedule/instruction.h`, and `src/schedule/schedule.cpp`, respectively.

A. How the schedule enters the simulator

The schedule is constructed before simulated time begins. In `sim_main.cpp`, the preprocessing phase accepts one of three front ends: a literal schedule YAML, a simple GEMM workload YAML that is expanded by the Tiler, or a LLaMA workload YAML that is expanded by `src/schedule/llama_schedule.cpp`. The LLaMA path reads a compact `LlamaScheduleConfig`, fills defaults such as `head_dim = hidden_dim / num_q_heads` and `gqa_group_size = num_q_heads / num_kv_heads`, validates that the dimensions are consistent, and then calls `build_llama_schedule`. The output is the same `Schedule` object used by hand-written tests, so generated and hand-written workloads share the same downstream simulator. The preprocessing path is implemented in `apps/sim_main.cpp`; LLaMA is implemented

in `src/schedule/llama_schedule.cpp`, and YAML keys are read in `src/schedule/llama_schedule.cpp`.

After the builder returns, `Tiler::expand_gemm_subtiles` checks whether any GEMM exceeds the configured array dimensions and rewires it into smaller array-sized operations when needed. The resulting graph is passed to the Scheduler, which builds dependency counters and successor lists. At runtime, an instruction becomes live when its dependency counter reaches zero; the registered operation handler reserves a unit pool and pushes an `OP_START` event into the event engine. Thus the schedule controls ordering and overlap, while the unit code controls latency. `apps/sim_main.cpp` is the main caller, with the GEMM expansion living in `src/schedule/tiler.cpp`. The scheduler’s dependency counters, successor lists, launch rule, and issue rule are in `src/schedule/scheduler.cpp`.

B. Schedule modularity

The LLaMA builder is organized as small append routines for model fragments: embeddings, RMSNorm, RoPE, tiled GEMMs, FlashAttention-2, the SwiGLU MLP, residual adds, the output head, softmax, sampling, and decode feedback. Each routine emits ordinary instructions with explicit dimensions.

For example, changing `hidden_dim`, `intermediate_dim`, `num_layers`, `num_q_heads`, `num_kv_heads`, `tile_rows`, or `tile_cols` changes the generated graph without changing the simulator. This is more useful than a fixed LLaMA-3 trace: the schedule builder is a workload generator parameterized by model shape and serving mode. The parameter fields exposed to the builder are declared in `src/schedule/llama_schedule.h`, and append routines for tiled GEMMs and vector phases are in `src/schedule/llama_schedule.cpp`.

The builder supports attention, layer, prefill, decode, and `prefill_decode` modes. This lets a designer isolate one attention block, one transformer block, prompt processing, one decode step, or a full prompt-plus-generation serving path. The same abstraction could also describe architectures beyond LLaMA-3. A model with different head counts, a different feed-forward expansion, a different tile shape, or a different KV-cache policy can be emitted by changing the workload YAML or by adding a new append routine that emits the same instruction vocabulary. If the model requires a new primitive, such as a sparse GEMM or a different normalization, it can be added through the operation registry as a new `op` string and handler rather than by modifying the scheduler. Mode dispatch is implemented in `src/schedule/llama_schedule.cpp`. The operation registry interface is in `src/schedule/op_registry.h`,

and the built-in operation registration pattern is in `src/schedule/op_handlers.cpp`.

The implementation also has host-side modularity features. In `--no-trace` mode the builder enters a minimal metadata path: it drops human-readable labels and symbolic buffer names while preserving numeric parameters such as M , K , N , row counts, column counts, and lengths. Since the timing model reads these numeric parameters, the reduced schedule is timing-equivalent but much smaller. This matters for experiments where a full prefill schedule contains millions of instructions. The minimal-mode filter is enabled from the command path in `apps/sim_main.cpp`.

C. The implemented LLaMA-3 workload

The implemented workload follows the LLaMA-3 decoder block: token embedding, RMSNorm, self-attention with RoPE and KV cache, a residual add, RMSNorm, a SwiGLU MLP, and a second residual. [1], [2] After the layer stack, the schedule emits the final output path through the language-model head, logits softmax, top-1 sampling, and token feedback for the next decode step. GEMMs are assigned to the systolic pool; RMSNorm, row reductions, exponentials, SiLU, accumulations, residual adds, softmax, and sampling are assigned to vector cores; transposes, copies, initialization fills, and stage-slot releases are assigned to access cores; HBM traffic and operand staging are assigned to DMA. The 8B workload file sets the real LLaMA-3 scale used in our experiments: 32 layers, hidden size 4096, 32 query heads, 8 KV heads, head dimension 128, intermediate size 14336, vocabulary size 128256, BF16 values, and a 2048-token prompt with one decode step. Layer-stack construction is in `src/schedule/llama_schedule.cpp`; The 8B workload constants are in `workloads/llama_prefill_decode_8B.yaml`.

D. FlashAttention-2 schedule

The attention schedule implements a tiled FlashAttention-2 forward pass. [3], [4] For each query tile and KV tile, the builder emits the inner sequence: stage Q , compute $S = QK^T$, scale scores, optionally apply a causal mask, compute row maxima, update the online maximum m , compute $P = \exp(S - m)$, update the running row sum l , rescale the running output accumulator O , compute PV , and accumulate into O . After all KV tiles for a query tile are consumed, the builder emits normalization by l and a log-sum-exp output.

`tile_rows` and `tile_cols` change the query and KV blocking. `causal_block_skip` toggles the causal block-skip path: fully future KV tiles are not emitted at all, diagonal tiles receive a `causal_mask` operation, and fully past tiles pass through without a mask. More invasive FlashAttention experiments, such as changing the inner algorithm order or adding a new score transformation, would require editing the builder or registering new vector operations, but would

still use the same simulator interface. The causal block-skip and conditional-mask schedule logic is implemented in `src/schedule/llama_schedule.cpp`.

E. KV cache and GQA

KV caching is modeled as explicit reads and writes rather than as an implicit attention shortcut. [5] During prefill, projected and RoPE-transformed keys and projected values are written to named cache ranges. During decode, the newly generated token appends its own K/V range, and attention reads all live ranges needed by the current token. Cache names include layer, KV head, tensor kind, page, block, and token range, which makes the address scheme consistent between prefill writes, decode appends, and decode reads. The KV-cache naming, HBM-versus-SRAM movement, prefill/decode writes, and attention reads are implemented in `src/schedule/llama_schedule.cpp`.

The cache location is configurable. The schedule also supports `kv_prefetch: double_buffer`, `kv_stage_buffers`, `kv_cache_block_tokens`, and `kv_cache_eviction_policy`. The double-buffered path alternates staging slots and inserts `kv_stage_release` events before a slot is reused, which lets the next K/V tile be fetched before the current tile has finished all compute. The `spill_to_hbm` policy allows experiments where only hot cache pages remain SRAM-resident while older pages pay HBM traffic. The details are implemented in `src/schedule/llama_schedule.h` and `src/schedule/llama_schedule.cpp`.

Grouped-Query Attention is represented by loop structure and dimensions. [6] The builder loops over KV heads outside the query-head loop. For LLaMA-3-8B, 8 KV heads serve 32 query heads, so each KV head feeds a group of 4 query heads. [2] The K and V projections therefore produce `num_kv_heads × head_dim` columns rather than `num_q_heads × head_dim` columns, and each staged K/V tile is reused by all query heads in its group. Because the reuse is expressed in the schedule graph, it appears in both the emitted memory operations and the MAC and HBM counters.

F. Serving environment and design-space use

The simulated serving environment is for a single-user, single seed query simulation. A `prefill_decode` run processes one prompt, writes its KV cache, then performs `generation_steps` serial decode iterations where each generated token is fed back into the next step. This models time-to-first-token and single-stream decode throughput. It does not currently model a multi-user server with request arrivals, batching, preemption, or queueing. That limitation is mostly at the workload-generation layer: multiple users could be represented as multiple request DAGs sharing the same hardware pools, with extra schedule policy deciding whether their prefill and decode nodes interleave or batch.

TABLE I
SIMULATOR PERFORMANCE ACROSS OPTIMIZATION STAGES

Stage	Time	RAM	Notes
Original	5 s/layer	>9 GB	$O(n^2)$ scan; OOM at Layer 4
Partial fix	~10 min	9 GB	Worst hashmaps removed
All hashmaps to vecs	~2 min	6 GB	Session start baseline
Final	35 s	2.4 GB	Intern codes; tiler pre-built; reserves

This schedule interface also makes the simulator useful for fast Pareto-frontier search. A user can hold a hardware YAML fixed and sweep model dimensions, GQA ratios, KV-cache locations, tile sizes, and prompt lengths to find the largest model that meets a latency or decode-throughput target. Conversely, a user can hold the workload fixed and sweep array size, unit counts, HBM bandwidth, DMA channels, SRAM size, and vector width. The sweep script’s hardware/workload cross product and CSV schema are in `compare.py`; the same file contains the parser plus derived KPI computation.

IV. IMPLEMENTATION, EVALUATION, AND RESULTS

A. Performance Engineering

The initial simulator was $O(n^2)$: `try_issue()` linearly scanned all instructions on every dispatch, causing Layer 1 to take 5 s of wall time with full runs still incomplete after 15 minutes. Three targeted changes achieved a 35 s run and 2.4 GB peak RAM for the LLaMA-3-8B model under our default HW and SW configs. (Table I): (1) all `std::string` fields for operation names, labels, and parameter keys replaced with 1–2 byte integer intern codes, shrinking the instruction struct from 152 B to 56 B; (2) the tiler decomposer moved outside the simulation loop so it runs once at build time rather than per-dispatch; (3) vectors pre-reserve capacity at construction, eliminating reallocation. Hardware model outputs are bit-identical before and after all changes.

B. Hardware Configurations

Table II summarizes the two preset YAML configurations. The **datacenter** target models an H100-class accelerator; the **edge** target models a mobile SoC (Apple M-series / Snapdragon-class NPU). The ridge point is the MACs/byte at which the workload transitions from memory-bound to compute-bound. Decode arithmetic intensity is ≈ 0.5 MACs/byte at `bs=1`, well below both ridge points (626 and 123 MACs/byte respectively), so the datacenter’s 170 $\times$ compute advantage is almost entirely wasted for this workload. This case study covers the 1B and 8B model configurations; the 70B workload results are available in the datacenter sweep CSV [7] but could not be run on the edge device, which exceeded the 16 GB pod memory budget.

C. Sweep Methodology and Roofline

All sweeps are driven by two Python scripts. `sweep_safe.py` handles the 1B-model groups and `sweep.py` handles the Group 6 calibration. Both generate YAML configuration pairs, invoke `sim_main` once per

TABLE II
HARDWARE CONFIGURATION COMPARISON

Parameter	Datacenter	Edge	Ratio
Systolic array	256 $\times$ 256, 8 units	32 $\times$ 32, 1 unit	64 $\times$ per array
Peak compute	2,097 TFLOPS	12.3 TFLOPS	170 $\times$
Memory BW	3.35 TB/s (HBM3)	0.10 TB/s (LPDDR5X)	33.5 $\times$
Ridge point	626 MAC/B	123 MAC/B	5.1 $\times$
On-chip SRAM	16 MB	1 MB	16 $\times$
Clock	2.0 GHz	1.5 GHz	1.3 $\times$

TABLE III
EXPERIMENT PLATFORM SPECIFICATIONS

Parameter	Value
CPU model	AMD EPYC 7662, 2 sockets $\times$ 64 cores
Logical cores (node)	256
Allocated cores (pod)	4 (<code>launch.sh -c 4</code>)
Allocated RAM (pod)	16 GB (<code>launch.sh -m 16</code>)
GPU	None (<code>launch.sh -g 0</code>)
OS	Debian 6.1, Linux 6.1.0-44-amd64
Memory guard	8 GB per run (<code>sweep_safe.py</code>)

configuration with `--no-trace`, parse the CSV output, and aggregate results. No recompilation is required between runs. Experiments ran on a DSMLP¹ pod from an AMD EPYC 7662 node; full system specifications and pod allocation are listed in Table III. `sweep_safe.py` enforces an 8 GB memory guard per run; runs exceeding the limit are logged as `FAILED` rows rather than crashing the sweep. Each simulation run emits a richer set of outputs than those reported here, including raw cycle counts, per-unit utilization for all four hardware pools, roofline efficiency, achieved TFLOPS, and the FFN/attention MAC split; the full metric definitions are documented in [8].

All five design-space sweep groups (Sections IV-D–IV-H, summarised in Appendix A) use the 1B model configuration (16 layers, 32Q/8KV heads, $d=2048$, maximum sequence length of 8192), which ran successfully on every configuration studied. The 8B workload triggered OOM failures on several sweep groups under the 16 GB pod constraint, preventing a complete sweep. We therefore report 1B results throughout, as they capture all architectural phenomena of interest: memory-bound decode, super-linear prefill scaling, GQA trade-offs, and the Pareto frontier, at a model scale that completes reliably. On a system with ≥ 32 GB available RAM the 8B workload runs without modification. The Group 6 calibration (Section IV-I), which requires the 8B model to match published chip references, was run separately with `sweep.py` using a dedicated 32-token decode pass.

Arithmetic intensity is measured empirically by the simulator as $total_MACs/total_HBM_bytes$ and is a workload property set by model dimensions and tiling, independent of hardware choice. Decode-only runs cluster at ≈ 0.5 MACs/byte at batch size 1: each step loads the full weight matrix from

¹UC San Diego Data Science and Machine Learning Platform

HBM to generate one token, leaving almost no compute reuse. The combined prefill+decode forward pass (2048-token prompt + 1 decode step at batch size 1) rises to 62.2–63.0 MACs/byte because prefill reuses each weight tile across all 2048 input tokens, dramatically increasing compute per HBM byte. While a theoretical untiled prefill would yield higher reuse, hardware SRAM limits require tiling across the token dimension, forcing weights to be re-fetched across token chunks and bounding the empirical intensity. Both values fall below the ridge point of the smallest chip (123 MACs/byte on the edge SoC), placing all configurations in the memory-bound regime.

D. Group 1: Compute Knobs

Array size (1a). Prefill TTFT drops sharply from 64×64 to 128×128 as the array exits the compute-bound regime (Fig. 2). From 128 to 256 TTFT is nearly flat; at 512×512 it increases slightly because the fill term $((r-1)+(c-1)=1022$ cycles; full equations in Appendix A) consumes a larger fraction of per-tile cost with no compensating gain. With FA2 tiles fixed at 256×256 , the 512×512 array is underutilized. Decode TPS peaks at 128×128 and declines for larger arrays: with $M=1$ (one new token per decode step) the GEMM latency is dominated by weight-load cycles rather than streaming compute, so extra PEs only increase the fill overhead.

Unit count (1b). Sweeping 1–4 systolic units delivers sub-linear decode gains: $23 \rightarrow 49$ tok/s ($2.1 \times$) from a $4 \times$ compute increase (Fig. 3). Systolic utilization falls from 86% to 46%, confirming that a $4 \times$ compute increase yields only $2.1 \times$ throughput gain as the memory wall absorbs the remaining headroom.

Bidirectional MACs (1c). Enabling bidirectional operation halves the fill term and reduces weight-load-to-compute ratio with no additional area (Fig. 4): prefill TTFT drops 18% ($507 \rightarrow 415$ ms) and decode TPS rises 30% ($27.1 \rightarrow 35.2$ tok/s). Bidirectional operation should be default-on for any systolic design.

E. Group 2: Memory Bandwidth

Bandwidth was swept across 0.5, 1.0, 2.0, and 3.35 TB/s (Fig. 5). TTFT improves from 469 to 406 ms (13% total) and decode TPS rises from 31.1 to 35.8 tok/s (15% total) over the full $6.7 \times$ bandwidth increase. Both metrics improve steeply from 0.5 to 1.0 TB/s, then largely flatten above 2.0 TB/s as other fixed overheads dominate: vector operation cycles (softmax, RMSNorm), DMA latency terms that do not scale with bandwidth, and SRAM staging time. Bandwidth investment helps most when the system is in the sub-1 TB/s regime; beyond that threshold, gains are marginal under a single-node single-user model.

F. Group 3: Workload Knobs

Prompt length (3a). TTFT scales super-linearly: 67 ms at 128 tokens, 116 at 512, 434 at 2048, and 2144 ms at 8192 (Fig. 6). This is the signature of $O(seq^2)$ attention cost. The FFN/attention MAC split shifts from 82%/18% at 128 tokens

to almost 65%/35% at 8192. A system designed for short context will find its bottleneck has moved entirely by the time context reaches 8K.

FA2 tile size (3b). TTFT is flat across tile sizes 128, 256, and 512 (417, 415, 425 ms). (Fig. 7) Tile size does not change the MAC count, only the number of SRAM accesses. A 128-row tile on the 256-row array leaves half the rows idle per iteration and pays the fill term twice as often, adding roughly 0.4% overhead. The 256-row tile is the natural fit; tile size is otherwise a second-order knob.

Decode context length (3e). Results captured through generation of 32 tokens. TPS falls monotonically from 38.3 to 26.4 tok/s (31% reduction) as context grows from 512 to 8192 tokens (Fig. 8, left). The right panel shows the mechanism: HBM traffic per token grows from 2.49 to 2.74 GB/token as the KV cache expands. Each decode step must reload the full KV cache from HBM, so throughput degrades proportionally with context length. Long-context decode is a memory-intensive problem that compounds with length.

G. Group 4: GQA Sweep

KV-head count was swept from 1 (MQA) to 32 (MHA), as shown in Fig. 9. Prefill HBM traffic increases monotonically: MQA moves 33.2 GB vs. MHA 38.4 GB (16% increase). Decode TPS barely moves: 35.3 at MQA vs. 34.5 at MHA, only a 2.4% change across the full range. Decode is bottlenecked by weight reads; even in MHA the KV cache is a small fraction of total HBM decode traffic. GQA is worth adopting for prefill efficiency and KV-cache footprint reduction, but will not rescue decode throughput. Results captured in both prefill+decode and decode(32 tokens) only modes.

H. Group 5: Pareto Frontier

A 3×3 grid sweeps array sizes $\{128, 256, 512\}$ against HBM bandwidths $\{1.0, 2.0, 3.35\}$ TB/s (Fig. 10). Across both prefill and decode, a smaller array with higher bandwidth consistently dominates a larger array with lower bandwidth. Specifically, 128×128 at 3.35 TB/s achieves 38.3 decode TPS vs. 29.6 for 512×512 at 3.35 TB/s; the oversized array’s fill term hurts even at equal bandwidth. The balanced operating point is 256×256 at 2.0 TB/s.

I. Calibration Against Published Hardware

The simulator was calibrated against H100 SXM5, A100 SXM4, and TPUv4 hardware configurations using the LLaMA-3-8B model in pure decode mode (`generation_steps=32`, `batch = 1`). The primary calibration metric used is *bytes per output token* (HBM bytes read per generated token), since P3 correctly models the weight volume fetched per decode step but does not model the bandwidth-efficiency mechanisms of real hardware (hardware prefetch pipelines, multiple memory controllers, fused memory requests). A secondary comparison of decode throughput (TPS) is included for context.

Table IV reports results for the two successful runs. For H100 SXM5 and A100 SXM4, the simulated bytes-per-token (15.28 GB) matches the reference value of 16.27 GB

TABLE IV

CALIBRATION AGAINST PUBLISHED HARDWARE (LLAMA-3-8B, BATCH=1, GENERATION_STEPS=32). B/TOK = HBM BYTES PER OUTPUT TOKEN (PRIMARY CALIBRATION METRIC); ERR = (sim - ref)/ref; BOTH RUNS PASS THE $\pm 20\%$ TARGET (✓). SIM TPS IS LOWER THAN PUBLISHED BECAUSE THE SYSTOLIC GEMV MODEL DOES NOT CAPTURE HARDWARE MEMORY-STREAMING EFFICIENCY (SEE TEXT). SDB = STAGE DOUBLE-BUFFER; BASE AND SDB ROWS ARE IDENTICAL, CONFIRMING THE BOTTLENECK IS TILE-SCHEDULING OVERHEAD.

Chip	Sys%	Bytes / Token (primary metric)				Decode TPS		
		Sim (GB)	Ref (GB)	Err%	Pass?	Sim	Pub.	Bound
H100 SXM5 [9]	82.1	15.28	16.27	-6.1	✓	13.9	120	memory
A100 SXM4 [10]	77.6	15.28	16.27	-6.1	✓	9.3	70	memory
TPUv4 [13]	—	OOM (exit 137) — excluded				—	43	—

within -6.1% , satisfying the $\pm 20\%$ calibration target [9] [10]. The reference is computed as the sum of all BF16 weight parameters (16.00 GB) plus the KV-cache footprint for a 2048-token context (0.27 GB), following standard LLaMA-3 weight accounting [11]. Enabling stage double-buffering (SDB) produces identical results for both chips, confirming that the decode bottleneck is tile-scheduling overhead rather than DMA pipeline stalls.

Simulated TPS is $8.6\times$ below published for H100 (13.9 vs. 120) and $7.5\times$ below for A100 (9.3 vs. 70) [12]. This gap arises because autoregressive decode at batch = 1 is a vector-matrix multiply (GEMV, $q_len = 1$), which forces one-row tiles through the 256×256 systolic array (0.4% row utilisation per tile) with full tile-scheduling overhead at every weight block. Real accelerators achieve $\approx 58\%$ HBM bandwidth utilisation for this workload through hardware-optimised streaming and out-of-order memory controllers; P3 achieves $\approx 6\%$ because it models a single DMA channel with serialised tile dispatch. The simulator is accordingly intended for sensitivity analysis and relative comparison rather than absolute throughput prediction.

The TPUv4 run terminated with an out-of-memory signal (exit code 137) before producing output and is excluded from Table IV. The TPUv4 configuration uses four 128×128 MXUs with 32 chained sequential decode steps, producing a schedule roughly $4\times$ larger than the H100 case; the process was killed by the OS before the event engine completed. A reduced generation_steps or a per-step trace truncation would be required to obtain results for this chip.

J. Datacenter vs. Edge Case Study

The two preset configurations (Table II) were run on the 1B and 8B models to quantify the end-to-end deployment gap. Table V reports the results. The decode gap ($9.7\text{--}12.3\times$) is far smaller than the raw compute gap ($170\times$) and is more strongly correlated with the memory bandwidth ratio ($33.5\times$); both chips stall on HBM and the edge chip simply stalls longer per byte. The prefill gap is larger ($25\times$ for 1B) because prefill processes 2048 tokens per forward pass, amortizing fixed overheads (DMA latency, vector-op cycles) across a much larger computation, so the bandwidth ratio contributes more to the total runtime gap. Both phases remain memory-bound

TABLE V

DATACENTER VS. EDGE PERFORMANCE (LLAMA-3, SINGLE-STREAM). THE DECODE SPEEDUP ($9.7\text{--}12.3\times$) TRACKS THE HBM BANDWIDTH RATIO ($33.5\times$), NOT THE PEAK-COMPUTE RATIO ($170\times$).

Model	Phase	Datacenter	Edge	Speedup
1B	Decode (tok/s)	3288	338	$9.7\times$
1B	Prefill ($\mu\text{s}/\text{tok}$)	47.9	1,202.8	$25.1\times$
8B	Decode (tok/s)	723	58	$12.3\times$
8B	Prefill ($\mu\text{s}/\text{tok}$)	312.9	8176.7	$\approx 26.6\times$

TABLE VI

SELF-CONSISTENCY VALIDATION AGAINST HAND DERIVATIONS

Quantity	Analytical	Simulator	Err.
Single-head MACs/layer	7.046×10^8	7.046×10^8	0.00%
Full 8B prefill MACs	1.488×10^{13}	1.488×10^{13}	$<0.01\%$
GQA attn HBM bytes	1.342×10^9	1.343×10^9	0.08%
Implied param. count	8.030×10^9	8.030×10^9	$<0.01\%$
FA2 causal skip	$1.7\text{--}1.8\times$	$1.88\times$	in band

on both chips. For the 8B model, the edge device runs out of on-chip memory beyond 512 tokens, bounding the prefill comparison at that context length. For the decode mode, 32 tokens were generated.

K. Self-Consistency Validation

Every MAC total matches independent hand derivations to within 0.01% (Table VI). The implied parameter count is 8.030B, matching the published LLaMA-3-8B figure exactly. The causal FA2 path cuts attention-kernel instruction count by $1.88\times$ at 2048 tokens, slightly above the FlashAttention-2 paper's stated $1.7\text{--}1.8\times$ band [3]. The end-to-end FA2 speedup at 2048 tokens is $1.37\times$ rather than $1.88\times$ because QKV and output projections still dominate total runtime and are unaffected by the causal skip.

V. CONCLUSION

P3 is a C++17 event-driven, timing-only simulator for heterogeneous LLM inference accelerators. Its four-layer architecture (operation registry, unit pool, dependency-graph schedule, and YAML configuration) allows any single design axis to be swept in isolation, making it a practical tool for pre-RTL design-space exploration. A full LLaMA-3-8B forward pass comprising roughly 11.6 million scheduled operations completes in 35 s on commodity hardware.

The central quantitative result is a concrete answer to the compute-versus-memory question for LLM inference. LLM decode is memory-bound on every hardware configuration studied. The $26.9\times$ throughput gap for the 8B model between the datacenter and edge configurations tracks the $33.5\times$ HBM bandwidth ratio far more closely than the $64\times$ array-area ratio or the $170\times$ peak-compute ratio. Decode arithmetic intensity ($\approx 62\text{--}63$ MACs/byte) is a workload property set by model dimensions and tile structure; it cannot be improved by adding

compute, and it lies below the ridge point of every chip studied.

Three design takeaways follow directly from the sweep results.

1. Bandwidth dominates decode. A 128×128 array at 3.35 TB/s achieves higher decode TPS than a 512×512 array at any bandwidth studied. For a fixed silicon budget, memory bandwidth is a better investment than compute area beyond a modest threshold.

2. Prefill and decode respond to hardware knobs in opposite directions. Larger arrays improve prefill TTFT up to the compute-bound threshold and hurt decode TPS through the fill-term overhead. GQA reduces prefill HBM traffic by 13% but changes decode TPS by only 2.4%. Bandwidth investment is the only lever that consistently helps both phases.

3. Long-context compounds the memory wall. From 512 to 8192 context tokens, decode TPS falls 31% and HBM traffic per token grows from 2.48 to 2.74 GB. The bottleneck changes character with scale, and a simulator like P3 makes these regime transitions visible before any silicon is committed.

The original P3 simulator was hardwired for LLaMA: adding a new model, a new hardware unit, or changing how units connect required editing C++ source files directly. Concurrently with this work, an extended version was developed by N.Neelakanta [14] that makes all three of those things plug-and-play. New models drop in through a simple interface, new units register themselves without touching `sim_main`, and hardware connections are configured through YAML without any code changes, making it more flexible

On top of this, two directions remain as planned future work. First, the simulator currently models single-stream batch=1 inference only; extending to batching ($M > 1$ activations per decode step) and multiple concurrent users would expose the compute-bound regime that single-stream results never reach, where arithmetic intensity grows with batch size and the systolic array is actually stressed. This is the practically relevant regime for datacenter deployments and would make sweep results directly comparable to production throughput numbers. Second, a numerical data path would enable functional verification alongside cycle counting: adding a value field to instructions and computation to unit handlers would let the simulator confirm correctness, not just timing, which is a prerequisite for using P3 as a compiler backend or in an optimization loop. Weight residency across the row-tile loop is a smaller localized change that would reduce conservatism in reported HBM traffic and narrow the remaining calibration gap with published hardware numbers.

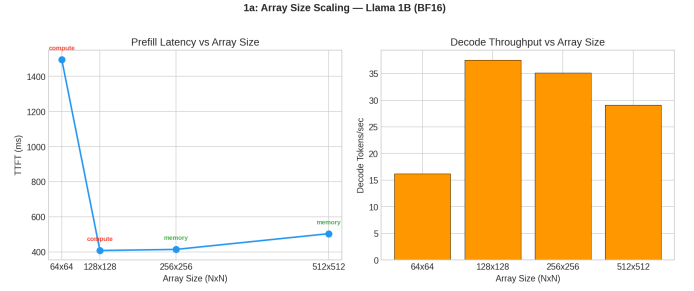


Fig. 2. Group 1a: prefill TTFT and decode TPS versus systolic array size (LLaMA-3 1B, BF16, FA2 tile = [256,256]). Decode peaks at 128×128 ; larger arrays hurt through the fill term. Prefill exits the compute-bound regime at 128×128 and is nearly flat thereafter.

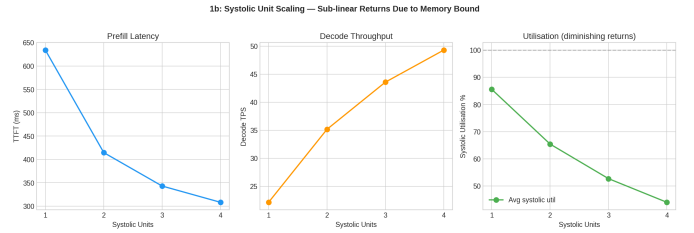


Fig. 3. Group 1b: prefill TTFT and decode TPS versus systolic unit count. Gains are sub-linear: $4 \times$ more units yields only $2.1 \times$ more decode throughput as the memory wall absorbs the headroom.

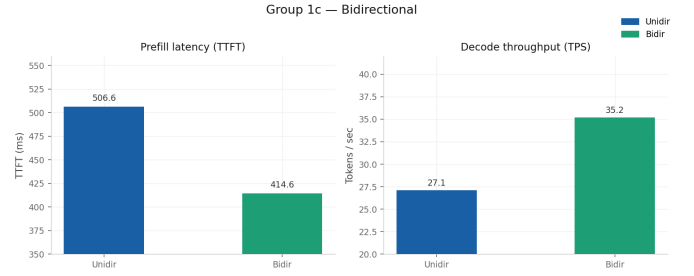


Fig. 4. Group 1c: effect of enabling bidirectional MACs. Prefill TTFT drops 18% and decode TPS rises 30% with no additional area, by halving the array fill term.

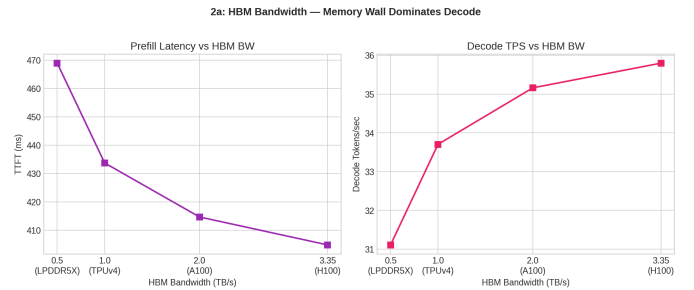


Fig. 5. Group 2a: prefill TTFT and decode TPS versus HBM bandwidth. Both metrics improve steeply from 0.5 to 1.0 TB/s then flatten as vector-op cycles and DMA latency overhead dominate above 2.0 TB/s.

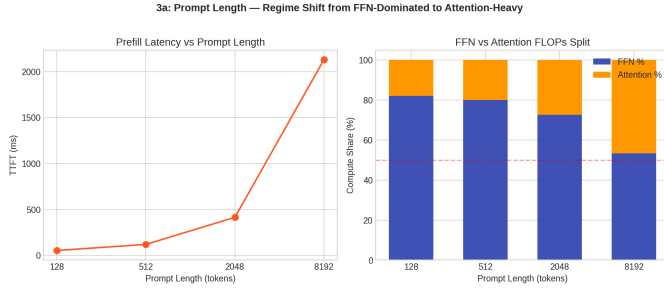


Fig. 6. Group 3a: prefill TTFT versus prompt length. Super-linear growth above 2048 tokens reflects the growing $O(seq^2)$ attention cost.

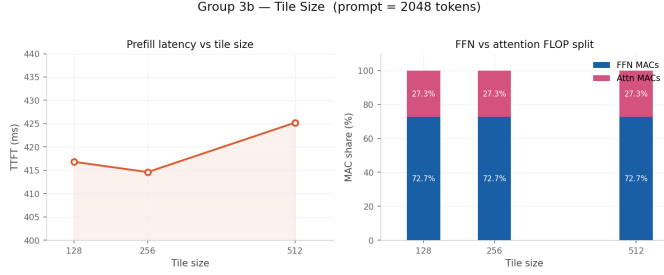


Fig. 7. Group 3b: prefill TTFT versus FA2 tile size at a fixed 2048-token prompt. TTFT is essentially flat across tile sizes 128, 256, and 512; the 256-row tile is the natural fit for the 256×256 array.

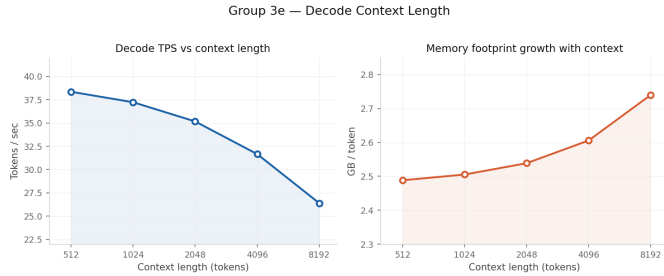


Fig. 8. Group 3c: decode TPS (left) and HBM traffic per token in GB (right) versus context length. TPS falls 31% from 512 to 8192 tokens as each decode step must reload a proportionally larger KV cache.

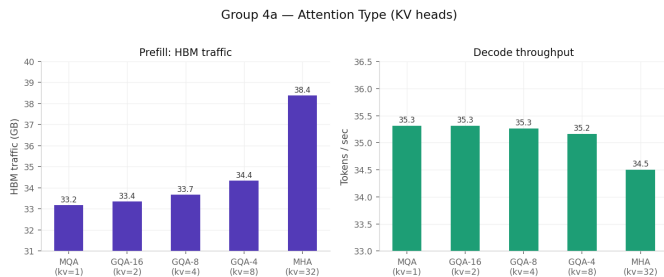


Fig. 9. Group 4a: prefill HBM traffic and decode TPS versus KV head count (MQA to MHA). Prefill traffic increases 16% from MQA to MHA; decode TPS changes by only 2.4%, confirming decode is bottlenecked by weight reads, not the KV cache.



Fig. 10. Group 5: Pareto frontier over systolic array size and HBM bandwidth. A smaller array at higher bandwidth consistently dominates a larger array at lower bandwidth on both prefill TTFT and decode TPS.

APPENDIX

Table VII lists the ten sweep groups reported in this paper; the full configuration reference for all available sweep groups is documented in [15].

TABLE VII
SWEEP GROUP CONFIGURATION REFERENCE

Grp	Axis Swept	Values	M	Fixed Default
1a	Array size ($N \times N$)	64, 128, 256, 512	1B	2 units, unidir, BW=2.0 TB/s
1b	Systolic unit count	1, 2, 3, 4	1B	256^2 array, unidir, BW=2.0 TB/s
1c	Bidirectional MACs	enabled / disabled	1B	256^2 , 2 units, BW=2.0 TB/s
2a	HBM bandwidth (TB/s)	0.5, 1.0, 2.0, 3.35	1B	256^2 , 2 units, unidir
3a	Prompt length (tokens)	128, 512, 2048, 8192	1B	256^2 , tile 256, BW=2.0 TB/s
3b	FA2 tile size	128, 256, 512	1B	256^2 , prompt=2048, BW=2.0 TB/s
3c	Decode context length (tokens)	512, 1024, 2048, 4096, 8192	1B	256^2 , BW=2.0 TB/s
4a	KV head count (GQA→MHA)	1, 2, 4, 8, 32	1B	256^2 , BW=2.0 TB/s
5	Array size × HBM BW (joint)	3×3 grid	1B	Pareto sweep over both axes
6	Calibration vs. published chips	H100, A100, TPUv4	8B	Matching published chip specs

Table VIII lists the latency for all hardware units. All quantities are in cycles unless otherwise noted. $r=rows$, $c=cols$, $wl=wload$ (cycles to load one K-block of weights), $bpc=bw \times channels$ (HBM bytes per cycle). M , K , N are the logical GEMM dimensions.

TABLE VIII
CLOSED-FORM LATENCY EXPRESSIONS FOR ALL SIMULATED HARDWARE
UNITS

Operation	Latency Formula	Notes
GEMM (weight-stationary, double-buf.)	$\lceil N/c \rceil \cdot (wl + \lceil K/r \rceil \cdot \max(wl, M) + fill)$	Standard case
GEMM (no double-buf.)	$\lceil N/c \rceil \cdot \lceil K/r \rceil \cdot (wl + M) + fill$	Legacy comparison
fill (unidirectional)	$(r-1) + (c-1)$	256 ² array: 510 cyc
fill (bidirectional)	$\lceil (r-1)/2 \rceil + \lceil (c-1)/2 \rceil$	256 ² array: 256 cyc
DMA (data ready)	$hbm_lat + \lceil bytes/bpc \rceil$	Data available to unit
DMA (pipelined)	channel $bytes/bw$ (else $lat + bytes/bw$)	Controls channel occupancy
SRAM (IBUF→PE)	stage $\lceil elements/r \rceil$	Wide operand bus
Vector	$passes \cdot \lceil e/simd \rceil + e_{ops} \cdot e_{lat} \cdot \lceil e/simd \rceil$	2nd term for exp/softmax
Access (copy/transpose)	$\lceil elements/bw \rceil$	Off critical path
Roofline (MAC/cyc)	peak $r \cdot c \cdot units \cdot (bidir?2:1)$	Array peak throughput
Roofline bound	$\max(\lceil MACs/pk \rceil, \lceil HBM_B/bpc \rceil);$ $pk=r \cdot c \cdot units \cdot (bidir?2:1)$	Determines bound regime

REFERENCES

- [1] H. Touvron et al., “LLaMA: Open and Efficient Foundation Language Models,” 2023.
- [2] Meta AI, “The Llama 3 Herd of Models,” 2024.
- [3] T. Dao, “FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning,” *arXiv preprint arXiv:2307.08691*, 2023. Available: <https://arxiv.org/abs/2307.08691>
- [4] T. Dao, D. Fu, S. Ermon, A. Rudra, and C. Ré, “FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness,” *NeurIPS*, 2022.
- [5] N. Shazeer, “Fast Transformer Decoding: One Write-Head is All You Need,” arXiv, 2019.
- [6] J. Ainslie et al., “GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints,” *EMNLP*, 2023.
- [7] T. Singh, A. Mohammad, and N. Neelakanta, “Datacenter Sweep Results,” GitHub, 2025. Available: https://github.com/AManOutOfTime/p3-tpu-event-model/blob/main/sweep/sweep_csv/datacenter.csv
- [8] T. Singh, A. Mohammad, and N. Neelakanta, “Simulator Outputs and KPI Reference Tables,” GitHub, 2025. Available: <https://github.com/AManOutOfTime/p3-tpu-event-model/blob/main/sweep/Simulator%20Outputs%20%26%20PPT%20KPI%20Reference%20Tables.pdf>
- [9] NVIDIA Corporation, “NVIDIA H100 Tensor Core GPU Architecture,” NVIDIA Whitepaper, 2022. Available: <https://resources.nvidia.com/en-us-tensor-core/gtc22-whitepaper-hopper>
- [10] NVIDIA Corporation, “NVIDIA A100 Tensor Core GPU Architecture,” NVIDIA Whitepaper, 2020. Available: <https://images.nvidia.com/aem-dam/en-zz/Solutions/data-center/nvidia-ampere-architecture-whitepaper.pdf>
- [11] Meta AI, “Meta Llama 3 Model Card,” GitHub, 2024. Available: https://github.com/meta-llama/llama3/blob/main/MODEL_CARD.md
- [12] G. Gerganov et al., “llama.cpp: Efficient LLM Inference in C/C++,” GitHub, 2024. Available: <https://github.com/ggerganov/llama.cpp>
- [13] N. P. Jouppi et al., “TPU v4: An Optically Reconfigurable Supercomputer for Machine Learning with Hardware Support for Embeddings,” in *Proc. 50th Annu. Int. Symp. Computer Architecture (ISCA)*, 2023. Available: <https://doi.org/10.1145/3579371.3589350>
- [14] N. Neelakanta, “P3 extended: plug-and-play unit interfaces and PagedAttention-compatible KV cache controller,” GitHub, 2025. Available: <https://github.com/cky26/p3-tpu-event-model>
- [15] T. Singh, A. Mohammad, and N. Neelakanta, “TPU Simulator Parameter Sweep Configuration Reference,” GitHub, 2025. Available: <https://github.com/AManOutOfTime/p3-tpu-event-model/blob/main/sweep/TPU%20Simulator%20Sweep%20Configuration%20Reference.pdf>
- [16] M. Milakov and N. Gimelshein, “Online Normalizer Calculation for Softmax,” arXiv, 2018.